

HANDS-ON EXERCISES

Setting Up Your Bitcoin Development Environment

- ★ We will run Bitcoin Core in **regtest** mode
- ★ Create wallets and generate addresses
- ★ Mine blocks and send transactions
- ★ Explore the blockchain with CLI commands

Prerequisite: Bitcoin Core installed

Bitcoin Day 1

SETUP STEP 1: CREATE DIRECTORY

```
# Create the Bitcoin configuration directory  
mkdir -p ~/.bitcoin
```

What this does:

★ Creates a hidden folder `.bitcoin` in your home directory

Bitcoin Day 1

SETUP STEP 2: CREATE CONFIG FILE

```
# Open the config file for editing  
nano ~/.bitcoin/bitcoin.conf
```

Or use any text editor:

- ★ `code ~/.bitcoin/bitcoin.conf` (VS Code)
- ★ `vim ~/.bitcoin/bitcoin.conf` (Vim)

SETUP STEP 3: ADD CONFIGURATION

Paste this into `bitcoin.conf`:

```
# Bitcoin Bootcamp Configuration  
regtest=1  
server=1  
daemon=1
```

SETUP STEP 4: MORE CONFIG

Continue adding to bitcoin.conf:

```
# Allow RPC from localhost  
rpccallowip=127.0.0.1  
rpcbind=127.0.0.1
```

```
# Set fallback fee for regtest  
fallbackfee=0.0001
```

```
# Index all transactions (needed for Day 2)  
txindex=1
```

Bitcoin Day 1

WHAT DOES THIS CONFIG DO?

Setting	Meaning
regtest=1	Use private test network
server=1	Enable RPC server
daemon=1	Run in background
rpcuser/password	Login credentials
rpcport=18443	Port for regtest RPC

SETUP STEP 5: START BITCOIN CORE

```
# Start the Bitcoin daemon  
bitcoind
```

You should see: (nothing - it runs in background)

```
# Verify it's running  
bitcoin-cli -retest getblockchaininfo
```

SETUP COMPLETE!

If you see this, you're ready:

```
{  
  "chain": "regtest",  
  "blocks": 0,  
  "headers": 0,  
  "bestblockhash": "0f9188f13cb7b2c71f2a335e3...",  
  ...  
}
```


EXERCISE 1: CHECK YOUR NODE

```
# Check blockchain info
```

```
bitcoin-cli -regtest getblockchaininfo
```

```
# Check network info
```

```
bitcoin-cli -regtest getnetworkinfo
```

Bitcoin Day 1

What do you see?

EXERCISE 2: CREATE WALLETS

```
# Create Alice's wallet
```

```
bitcoin-cli -regtest createwallet "alice"
```

```
# Create Bob's wallet
```

```
bitcoin-cli -regtest createwallet "bob"
```

```
# list all wallets
```

EXERCISE 3: GENERATE ADDRESSES

```
# Get address for Alice
```

```
bitcoin-cli -regtest -rpcwallet=alice getnewaddress
```

```
# Get address for Bob
```

```
bitcoin-cli -regtest -rpcwallet=bob getnewaddress
```

```
# Check Alice's balance
```

```
bitcoin-cli -regtest -rpcwallet=alice getbalance
```

EXERCISE 4: MINE BLOCKS

```
# Store Alice's address in a variable
ALICE=$(bitcoin-cli -regtest -rpcwallet=alice getnewaddress)

# Mine 101 blocks to Alice
bitcoin-cli -regtest generatetoaddress 101 "$ALICE"

# Check balance now
bitcoin-cli -regtest -rpcwallet=alice getbalance
```

Why 101? Coinbase maturity = 100 blocks

Result: Alice now has **50 BTC!**

Bitcoin Day 1

EXERCISE 5: SEND TRANSACTION

```
# Get Bob's address
```

```
BOB=$(bitcoin-cli -regtest -rpcwallet=bob getnewaddress)
```

```
# Send 10 BTC from Alice to Bob
```

```
bitcoin-cli -regtest -rpcwallet=alice sendtoaddress "$BOB" 10
```

```
# Check mempool (unconfirmed transactions)
```

```
bitcoin-cli -regtest getmempoolinfo
```

EXERCISE 5: CONFIRM IT

```
# Mine a block to confirm the transaction  
bitcoin-cli -regtest generatetoaddress 1 "$ALICE"
```

```
# Check Bob's balance  
bitcoin-cli -regtest -rpcwallet=bob getbalance
```

```
# Mempool should be empty now  
bitcoin-cli -regtest getmempoolinfo
```

Bitcoin Day 1

Result: Bob now has **10 BTC!**

EXERCISE 6: EXPLORE TRANSACTION

```
# List Alice's transactions
bitcoin-cli -regtest -rpcwallet=alice listtransactions

# Get raw transaction (replace TXID with actual ID)
bitcoin-cli -regtest getrawtransaction "TXID" true
```

Look for:

Bitcoin Day 1

EXERCISE 7: EXPLORE A BLOCK

```
# Get the latest block hash  
bitcoin-cli -regtest getbestblockhash
```

```
# Get block details (replace HASH)  
bitcoin-cli -regtest getblock "HASH" 1
```

Bitcoin Day 1

```
# Get full transaction data
```


EXERCISE 8: ADDRESS TYPES

Legacy (oldest)

```
bitcoin-cli -regtest -rpcwallet=alice getnewaddress "" "legacy"
```

Result: m... or n...

Native SegWit (recommended)

```
bitcoin-cli -regtest -rpcwallet=alice getnewaddress "" "bech32"
```

Result: bcrt1q...

Taproot (newest)

```
bitcoin-cli -regtest -rpcwallet=alice getnewaddress "" "bech32m"
```

Bitcoin Day 1

WHY SEGWIT MATTERS

Segregated Witness (2017 upgrade)



Before: Signatures inside transaction

★ Transaction ID could be changed
(malleability)

★ Limited block capacity



STOPPING BITCOIN CORE

```
# Stop the Bitcoin daemon gracefully  
bitcoin-cli -regtest stop
```

To restart later:

```
bitcoind
```

Your wallets and blockchain data are saved in

```
/ bitcoin/regtest /
```

DAY 1 SUMMARY

You learned:

- ★ Configure Bitcoin Core for regtest
- ★ Create wallets and generate addresses
- ★ Mine blocks and earn coinbase rewards
- ★ Send transactions between wallets
- ★ Explore blocks and transactions via CLI
- ★ Different address types and SegWit

Bitcoin Day 1

Tomorrow: Python coding with Bitcoin!

HOMework

★ Send 5 more transactions between Alice and Bob

★ Decode each transaction - understand inputs/outputs

★ Create wallet "charlie" and do 3-way transactions

★ Compare transaction sizes with different address types